

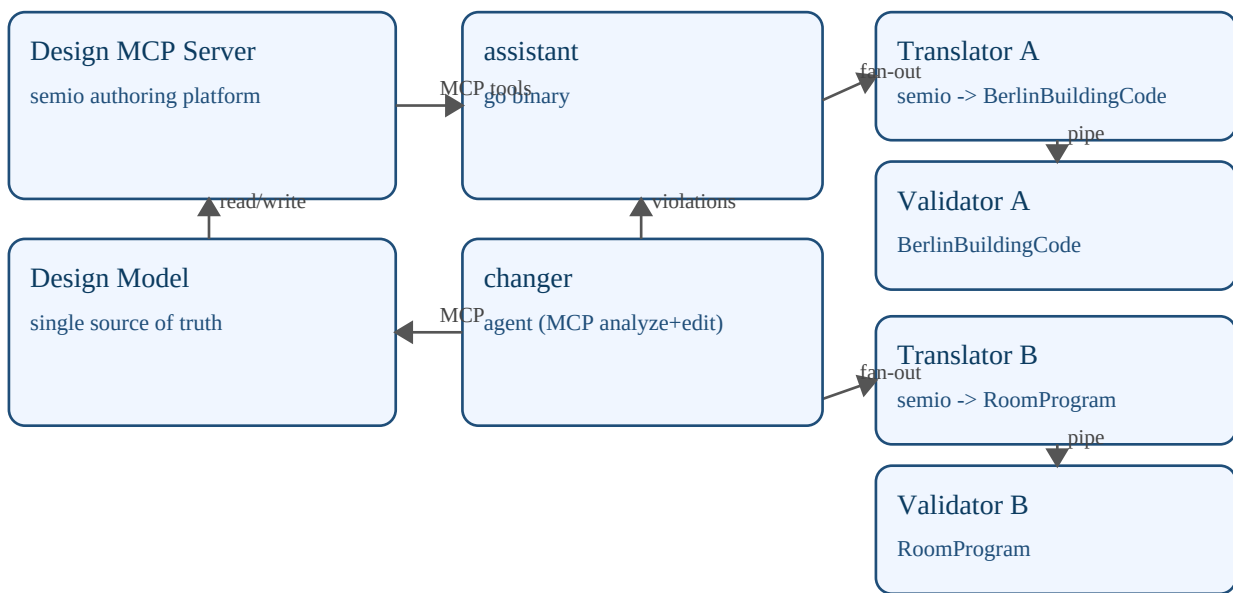
# Design Assistance Architecture Plan

Single-file Go implementation at [go/assistant/main.go](#)

## Scope

This document specifies a concrete architecture and implementation plan for a Go binary called **assistant**. The assistant orchestrates concurrent translation and validation of a single source **design** (authored in an MCP-enabled platform such as **semio**) against multiple compliance **targets** (e.g., **BerlinBuildingCode**, **RoomProgram**). When violations exist, an agent **changer** iteratively edits the design via the design MCP server and the cycle repeats until violations are resolved or iteration limits are reached.

## System context (example targets)



Key property: **translator output is streamed directly into its paired validator**. All translator+validator pairs run concurrently; validators start as soon as their translator finishes.

# 1. Goals and Non-goals

## Goals

- **Deterministic orchestration**: reproducible runs; clear timeouts and cancellation.
- **Concurrent fan-out** across targets; fast feedback by starting each validator immediately after its translator.
- **Structured reports** with rule → clause → measure hierarchy and stable IDs for diffing across iterations.
- **Iterative fixing** via a changer agent using MCP analyze tools + change tools.
- **Single-file implementation** in `go/assistant/main.go` with minimal dependencies.

## Non-goals

- Not a general workflow engine; orchestration is purpose-built for this assistance loop.
- Not a UI; the primary output is a machine-readable JSON report plus a human-readable summary.
- Not a full MCP client implementation; we assume an MCP Go client is available or shelling out is acceptable.

## 2. Domain Model

Validators enforce **rules**. Each rule contains **clauses**. A **violation** exists when at least one clause is not satisfied. A clause can emit one or more **measures** (observations/metrics) that justify the decision and support remediation.

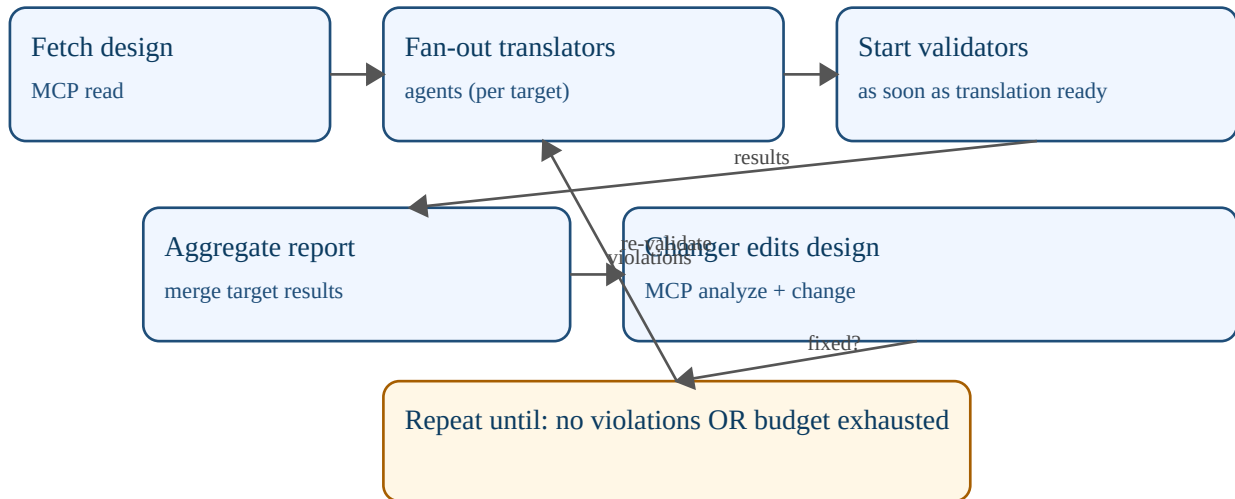
| Entity    | Purpose                                                 | Key fields (stable IDs recommended)                                             |
|-----------|---------------------------------------------------------|---------------------------------------------------------------------------------|
| Rule      | Compliance intent that can be evaluated                 | id, title, description, severity, version                                       |
| Clause    | Atomic check; failing clause produces violation         | id, ruleId, statement, evidenceRefs[]                                           |
| Measure   | Quantitative/qualitative observation to explain outcome | id, clauseId, name, value, unit, threshold, context                             |
| Violation | A failed clause with remediation hints                  | id, target, ruleId, clauseId, message, location, measures[], remediationHints[] |
| Report    | Aggregated results across targets and iterations        | designId, iteration, targets[], violations[], summary                           |

### ***Stable IDs***

Use IDs that remain stable across iterations (e.g., rule IDs from the target specification, clause IDs from validator source, and design element IDs from semio). This enables the changer to map violations back to concrete design elements and track whether an issue is truly fixed or merely moved.

### 3. Runtime Workflow

One run consists of **Translate** → **Validate** (per target, concurrently) → **Aggregate report** → **Change** (if needed) → repeat.



**Concurrency rule:** for each target, the assistant starts the translator and waits for its output. When a translator completes successfully, the assistant immediately spawns its paired validator with the translation as stdin. This enables long-running validators to overlap with other translations.

**Cancellation:** the assistant uses a root context with deadline; a failure in one target may either (a) fail-fast or (b) mark that target as errored and continue collecting other targets. Default recommendation: continue collecting to maximize signal, but stop the changer if report integrity is compromised.

## 4. Component Interfaces

This plan assumes each translator is an **agent** (invoked either as a local binary or via RPC), and each validator is a **binary** that accepts translated input on stdin and returns a structured JSON result on stdout.

### 4.1 Translator contract

Input: design reference (designId, optional snapshot/hash) plus target name. Output: a target-specific document.

```
// Translator stdout MUST be the full translated document.
// The assistant does not interpret the content; it only pipes it into the validator.
```

```
{
  "target": "BerlinBuildingCode",
  "format": "berlin_building_code/v1",
  "designId": "semio:project:abc",
  "generatedAt": "2026-01-23T10:15:00Z",
  "payload": { /* target-specific */ }
}
```

### 4.2 Validator contract

```
// Validator reads translator output from stdin.
// Validator stdout is a JSON envelope with rule/clause results and violations.
```

```
{
  "target": "BerlinBuildingCode",
  "validatorVersion": "1.4.2",
  "status": "ok", // ok | warn | error
  "rulesEvaluated": 128,
  "violations": [
    {
      "id": "bbc.R12.C3",
      "ruleId": "bbc.R12",
      "clauseId": "bbc.R12.C3",
      "severity": "high",
      "message": "Second egress route missing for Room 2.14",
      "location": { "designNodeId": "semio:node:room:2.14" },
      "measures": [
        { "id": "m1", "name": "egressRoutes", "value": 1, "unit": "count", "threshold": 2 }
      ],
      "remediationHints": [
        "Add a second compliant egress path or reclassify occupancy."
      ]
    }
  ]
}
```

#### **Streaming note**

To avoid buffering large translations in memory, the assistant can pipe translator stdout directly into validator stdin using `io.Pipe`. If the translator output is JSON, it can be streamed byte-for-byte; the validator is responsible for full parsing.

# 5. Go Implementation Plan (single file)

Everything lives in **go/assistant/main.go**. The file is organized into clearly separated sections.

| Section         | Contents                                                                              |
|-----------------|---------------------------------------------------------------------------------------|
| Config          | Targets, translator commands, validator commands, timeouts, iteration budget, logging |
| Types           | Report/Violation/Rule/Clause/Measure structs; TargetResult; ProcessResult             |
| Process runners | runTranslator(), runValidator(), pipeTranslatorToValidator()                          |
| Orchestrator    | runOnce() fan-out, aggregate(); runLoop() iterative changer cycle                     |
| Changer adapter | invokeChangerAgent() and MCP helpers (analyze+change)                                 |
| Main            | Parse flags/env, build registry, execute runLoop, write report artifacts              |

## 5.1 Target registry (example)

```
type TargetID string

const (
    TargetBerlinBuildingCode TargetID = "BerlinBuildingCode"
    TargetRoomProgram        TargetID = "RoomProgram"
)

type TargetSpec struct {
    ID                TargetID
    TranslatorCmd      []string // e.g. ["translator-semio-bbc", "--design", "{designId}"]
    ValidatorCmd       []string // e.g. ["validator-bbc", "--json"]
    TranslatorEnv      map[string]string
    ValidatorEnv       map[string]string
    MaxTranslateSec    int
    MaxValidateSec     int
}

var Targets = []TargetSpec{
    {ID: TargetBerlinBuildingCode, TranslatorCmd: []string{"translator-semio-bbc"}, ValidatorCmd: []string{"validator-bbc"}, TranslatorEnv: map[string]string{"designId": "123"}, ValidatorEnv: map[string]string{"format": "json"}, MaxTranslateSec: 30, MaxValidateSec: 10},
    {ID: TargetRoomProgram, TranslatorCmd: []string{"translator-semio-roomprogram"}, ValidatorCmd: []string{"validator-bbc"}, TranslatorEnv: map[string]string{"designId": "123"}, ValidatorEnv: map[string]string{"format": "json"}, MaxTranslateSec: 30, MaxValidateSec: 10},
}
```

## 5.2 Concurrency primitives

Use **errgroup.Group** with a root context. Each target runs as an independent goroutine. Within that goroutine:

- Start translator (exec.CommandContext). Capture stdout as a stream.
- When translator completes, immediately start validator with translator output as stdin (stream or buffered).
- Parse validator stdout JSON into TargetResult. Return result (or error) for aggregation.

## 6. Orchestration Pseudocode

```
// runOnce validates the current design state against all targets concurrently.
func runOnce(ctx context.Context, designRef DesignRef, targets []TargetSpec) (Report, error) {
    g, ctx := errgroup.WithContext(ctx)
    results := make(chan TargetResult, len(targets))

    for _, t := range targets {
        t := t
        g.Go(func() error {
            trCtx, cancel := context.WithTimeout(ctx, time.Duration(t.MaxTranslateSec)*time.Second)
            defer cancel()

            // 1) Translate
            translated, trMeta, err := runTranslator(trCtx, t, designRef)
            if err != nil { results <- TargetResult{Target: t.ID, Status:"error", Err: err}; return nil }

            // 2) Validate (starts immediately after translation)
            valCtx, cancel2 := context.WithTimeout(ctx, time.Duration(t.MaxValidateSec)*time.Second)
            defer cancel2()

            vr, err := runValidator(valCtx, t, translated)
            if err != nil { results <- TargetResult{Target: t.ID, Status:"error", Err: err}; return nil }

            results <- TargetResult{Target: t.ID, Status:"ok", Translator: trMeta, Validator: vr}
            return nil
        })
    }

    _ = g.Wait()
    close(results)
    return aggregate(designRef, results), nil
}

// runLoop repeats validate -> change -> validate until done or budget exhausted.
func runLoop(ctx context.Context, designRef DesignRef) (FinalOutcome, error) {
    for iter := 1; iter <= cfg.MaxIterations; iter++ {
        report, _ := runOnce(ctx, designRef, Targets)
        writeArtifacts(iter, report)

        if len(report.Violations) == 0 { return FinalOutcome{Status:"compliant", Iterations: iter}, nil }

        changeRes := invokeChangerAgent(ctx, designRef, report)
        if !changeRes.Modified { return FinalOutcome{Status:"stuck", Iterations: iter, LastReport: report}, nil }

        // If the design MCP supports versions/snapshots, update designRef to the new revision.
        designRef = changeRes.NewDesignRef
    }
    return FinalOutcome{Status:"budget_exhausted"}, nil
}
```

### **Error handling choice**

The pseudocode treats per-target failures as data (TargetResult{Status:error}) rather than aborting the whole run. This is usually preferable because the changer can still act on the violations from successful targets. If a target is critical, mark it as required and fail the iteration if it errors.

## 7. Changer Agent Plan

The changer receives the full report and attempts to fix violations via MCP. It uses two classes of MCP tools:

- **Analyze tools** to inspect the design model: find node IDs, properties, relationships, geometry/areas, etc.
- **Change tools** to edit the design: add/remove elements, adjust parameters, reroute connections, update metadata.

### 7.1 Suggested changer algorithm

- Group violations by (target, ruleId) and then by location.designNodeId.
- For each location group, fetch design context via MCP analyze tools (neighbors, constraints, dimensions).
- Select a remediation strategy using remediationHints + domain heuristics (e.g., add egress, widen corridor, rename room type).
- Apply changes with MCP change tools, recording a change log (operations, parameters, before/after).
- Re-run local analyses for impacted nodes to catch obvious regressions early (optional).
- Stop when: (a) no remaining actionable violations, (b) no progress in an iteration, or (c) max change operations reached.

### 7.2 MCP integration points (semio example)

Analyze:

```
- design.getNode(id)
- design.searchNodes(query)
- design.getGraphNeighborhood(id, depth)
- design.computeArea(id) / computeDistances(...)
- design.validateInternalConstraints(...)
```

Change:

```
- design.updateNode(id, patch)
- design.addNode(type, props, parentId)
- design.deleteNode(id)
- design.connect(a, b, kind)
- design.beginTransaction() / commit() / rollback()
```

#### *Iteration hygiene*

Prefer transactional edits: apply a batch of changes for a group of related violations, commit, then re-validate. If the changer detects it made the situation worse (e.g., new high-severity violations), it should rollback or revert and try an alternate strategy.



## 8. Report Artifacts and Outputs

The assistant writes two artifacts per iteration:

- **report.iter-N.json**: full machine-readable report with all target results and violations.
- **report.iter-N.md**: short human-readable summary for engineers/architects.

```
# Iteration 2 summary
```

```
Design: semio:project:abc@rev=17
```

```
Targets: BerlinBuildingCode (ok), RoomProgram (ok)
```

```
Violations: 3 total (2 high, 1 medium)
```

- [HIGH] bbc.R12.C3: Second egress route missing for Room 2.14 (node semio:node:room:2.14)
- [HIGH] bbc.R07.C1: Corridor width below minimum near Stair S2 (node semio:node:corridor:level2-A)
- [MED] rp.R03.C2: Room function label mismatch: "Office" must be "Workplace"

For downstream automation, the JSON report should include a **runId**, timestamps, and validator versions to support audit and reproducibility.

## Appendix A. main.go Skeleton (illustrative)

```
// go/assistant/main.go
package main
```

```
import (
    "bytes"
    "context"
    "encoding/json"
    "os/exec"
    "time"

    "golang.org/x/sync/errgroup"
)
```

```
// ... types omitted for brevity ...
```

```
func main() {
    // Parse config, set defaults, create root ctx with deadline, call runLoop.
}
```

This skeleton is intentionally incomplete; the earlier sections specify the behavior and constraints that the final code must implement.